🎭 Playwright

Documentation | API reference

Playwright is a framework for Web Testing and Automation. It allows testing Chromium, Firefox and WebKit with a single API. Playwright is built to enable cross-browser web automation that is **ever-green**, **capable**, **reliable** and **fast**.

	Linux	macOS	Windows
Chromium 140.0.7339.24	✅	✅	✅
WebKit 26.0	✅	✅	✅
Firefox 141.0	✅	✅	✅

Headless execution is supported for all browsers on all platforms. Check out system requirements for details.

Looking for Playwright for Python, .NET, or Java?

Installation

Playwright has its own test runner for end-to-end tests, we call it Playwright Test.

Using init command

The easiest way to get started with Playwright Test is to run the init command.

```
# Run from your project's root directory
npm init playwright@latest
# Or create a new project
npm init playwright@latest new-project
```

This will create a configuration file, optionally add examples, a GitHub Action workflow and a first test example.spec.ts. You can now jump directly to writing assertions section.

Manually

Add dependency and install browsers.

```
npm i -D @playwright/test
# install supported browsers
npx playwright install
```

You can optionally install only selected browsers, see install browsers for more details. Or you can install no browsers at all and use existing browser channels.

- Getting started
- API reference

Capabilities

Resilient • No flaky tests

Auto-wait. Playwright waits for elements to be actionable prior to performing actions. It also has a rich set of introspection events. The combination of the two eliminates the need for artificial timeouts - a primary cause of flaky tests.

Web-first assertions. Playwright assertions are created specifically for the dynamic web. Checks are automatically retried until the necessary conditions are met.

Tracing. Configure test retry strategy, capture execution trace, videos and screenshots to eliminate flakes.

No trade-offs • No limits

Browsers run web content belonging to different origins in different processes. Playwright is aligned with the architecture of the modern browsers and runs tests out-of-process. This makes Playwright free of the typical in-process test runner limitations.

Multiple everything. Test scenarios that span multiple tabs, multiple origins and multiple users. Create scenarios with different contexts for different users and run them against your server, all in one test.

Trusted events. Hover elements, interact with dynamic controls and produce trusted events. Playwright uses real browser input pipeline indistinguishable from the real user.

Test frames, pierce Shadow DOM. Playwright selectors pierce shadow DOM and allow entering frames seamlessly.

Full isolation • Fast execution

Browser contexts. Playwright creates a browser context for each test. Browser context is equivalent to a brand new browser profile. This delivers full test isolation with zero overhead. Creating a new browser context only takes a handful of milliseconds.

Log in once. Save the authentication state of the context and reuse it in all the tests. This bypasses repetitive log-in operations in each test, yet delivers full isolation of independent tests.

Powerful Tooling

Codegen. Generate tests by recording your actions. Save them into any language.

Playwright inspector. Inspect page, generate selectors, step through the test execution, see click points and explore execution logs.

Trace Viewer. Capture all the information to investigate the test failure. Playwright trace contains test execution screencast, live DOM snapshots, action explorer, test source and many more.

Looking for Playwright for TypeScript, JavaScript, Python, .NET, or Java?

Examples

To learn how to run these Playwright Test examples, check out our getting started docs.

Page screenshot

This code snippet navigates to Playwright homepage and saves a screenshot.

```
import { test } from '@playwright/test';

test('Page Screenshot', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await page.screenshot({ path: `example.png` });
});
```

Mobile and geolocation

This snippet emulates Mobile Safari on a device at given geolocation, navigates to maps.google.com, performs the action and takes a screenshot.

```
import { test, devices } from '@playwright/test';

test.use({
  ...devices['iPhone 13 Pro'],
  locale: 'en-US',
  geolocation: { longitude: 12.492507, latitude: 41.889938 },
  permissions: ['geolocation'],
})

test('Mobile and geolocation', async ({ page }) => {
  await page.goto('https://maps.google.com');
  await page.getByText('Your location').click();
  await page.waitForRequest(/.*preview\/pwa/);
  await page.screenshot({ path: 'colosseum-iphone.png' });
});
```

Evaluate in browser context

This code snippet navigates to example.com, and executes a script in the page context.

```
import { test } from '@playwright/test';

test('Evaluate in browser context', async ({ page }) => {
  await page.goto('https://www.example.com/');
  const dimensions = await page.evaluate(() => {
    return {
      width: document.documentElement.clientWidth,
      height: document.documentElement.clientHeight,
      deviceScaleFactor: window.devicePixelRatio
    }
  });
  console.log(dimensions);
});
```

Intercept network requests

This code snippet sets up request routing for a page to log all network requests.

```
import { test } from '@playwright/test';

test('Intercept network requests', async ({ page }) => {
  // Log and continue all network requests
  await page.route('**', route => {
```

```
    console.log(route.request().url());
    route.continue();
  });
  await page.goto('http://todomvc.com');
});
```

Resources

- Documentation
- API reference
- Contribution guide
- Changelog